

Cross-Rollup Arbitrage Detection and Simulation on Decentralized Exchanges

Grounded Audit of the Implemented Research Pipeline

March 27, 2026

Abstract

This report rewrites the project draft against the repository’s actual implementation rather than its intended end state. The codebase does *not* implement a deployable bridge-aware cross-rollup executor or a smart-contract arbitrage engine. Instead, it implements two complementary Python pipelines: (i) a *single-chain detection* pipeline that scans Arbitrum, Base, and Optimism swap logs and reconstructs realized arbitrage transactions at the transaction level, and (ii) a *simulation* pipeline that enumerates reserve-token cycles, replays them under updated Uniswap pool states, and estimates theoretical profitability under a deterministic synchronization rule.

The verified data stack is hybrid. The pool universe is seeded from Dune; reserve-token prices, initial pool states, and replay event histories are pulled from Allium; local MongoDB stores normalized pool updates and exported opportunities; and the only explicitly committed RPC provider brand is Ankr, used as a fallback in the Uniswap v3 liquidity updater. The protocol mechanics implemented in code are Uniswap v2 and Uniswap v3, but the profitable simulation exports committed in the repository are entirely Uniswap v3-based. The path builder constructs single-chain reserve-token cycles and pairwise cross-rollup spatial cycles of length up to five; it is therefore inaccurate to describe the repository as merely “triangle arbitrage” logic.

The latest committed output artifacts in the repository are dated **November 1, 2024 (UTC)**. On those stored exports, cross-chain simulation produces 43,199 windows and \$1.764M of total theoretical profit, while single-chain simulation produces 122,401 windows and \$890.98 of total theoretical profit. Realized single-chain detection outputs are far smaller in magnitude but economically more credible because they include transaction costs: Base contributes the largest total detected profit (\$27,610.34), Arbitrum contributes \$1,297.31, and Optimism contributes \$113.08. These results are informative, but they remain upper bounds on deployable cross-rollup performance because the simulation omits bridge settlement delay, inventory constraints, and adversarial competition.

1 Introduction

Liquidity fragmentation across Ethereum rollups can create temporary price discrepancies even when the assets are economically equivalent. In this repository, that problem is studied over Arbitrum, Optimism, and Base, with automated market maker mechanics drawn from Uniswap v2 and Uniswap v3 [1, 2]. The economically relevant cross-rollup friction is not just AMM slippage;

on OP Stack-style systems the transaction cost itself is two-part, combining L2 execution fees with an L1 data-availability component [6, 4]. Any serious empirical claim about arbitrage therefore has to distinguish carefully between *path profitability under synchronized states* and *profitability under executable cross-domain settlement*.

That distinction is the central purpose of this revision. The raw draft mixed implemented components with proposed extensions. After auditing the repository, the technically correct description is the following:

- the **detection/** directory implements *single-chain realized arbitrage detection* from observed swap logs;
- the **simulation/** directory implements *path construction and deterministic replay* for reserve-token cycles, including pairwise cross-rollup cycles;
- the repository does *not* contain a bridge model, a cross-domain executor, or smart contracts that close cross-rollup positions on-chain.

Accordingly, this report answers four concrete questions:

1. What data sources, chains, pool types, and reserve assets are actually active in the repository?
2. How do the detection and simulation algorithms operate in code?
3. What do the committed November 1, 2024 exports actually show?
4. Which claims should be treated as implemented results, and which belong in future work only?

2 Verified System Scope

The repository contains no Solidity contracts, no bridge executor, and no direct settlement engine. All implemented arbitrage logic lives in Python scripts. That matters because the project should be assessed as a research infrastructure for opportunity generation and replay, not as a production deployment stack.

2.1 Pool universe and reserve-token mapping

The pool universe is built by `data/download_top_pools.py` and `data/extract_pools.py`. The first script queries Dune query 4573557 through the Dune API [5]; the second filters the result to Uniswap pools with non-empty fee metadata and then keeps only unique pool addresses. The reserve-token seed set is hard-coded as

$$\mathcal{R} = \{\text{WETH}, \text{USDC}, \text{USDT}, \text{WBTC}, \text{DAI}\}.$$

Applying the repository’s own filtering logic to the committed `top_pools_over_100m.csv` yields:

- 57 unique Arbitrum pools: 55 Uniswap v3 and 2 Uniswap v2;
- 83 unique Base pools: 68 Uniswap v3 and 15 Uniswap v2;
- 17 unique Optimism pools: all 17 are Uniswap v3.

The reserve-token address mapping is not uniform across chains. WETH, USDC, and USDT exist on all three rollups; DAI exists on Arbitrum and Optimism only; WBTC is Arbitrum-only in the committed pool universe and therefore cannot anchor a cross-rollup mapping.

2.2 Verified data and infrastructure sources

The repository uses Dune and Allium as off-chain indexers, local MongoDB as the replay store, and chain RPC calls only where the Python code needs direct on-chain state or log access. Table 1 summarizes the verified source stack.

Table 1: Verified data and infrastructure sources in the committed repository.

Component	Scripts	Verified source	Technical role
Pool universe	<code>download_top_pools.py</code>	Dune query API [5]	Downloads query 4573557 and seeds the pool universe used by <code>extract_pools.py</code> .
Reserve-token prices	<code>download_token_prices.py</code>	Allium Explorer [3]	Pulls reserve-token price time series via query <code>Q1pvUM7bqJIa7bJRNMLk</code> .
Initial v2 state	<code>download_initial_pool_states.py</code>	Allium Explorer [3]	Pulls v2 reserves via query <code>hf7Ayf6BVo83ArAjQ0X1</code> .
Initial v3 state	<code>download_initial_pool_states.py</code>	Allium Explorer [3]	Pulls swap/mint/burn history via query <code>Y1y4Q4pbNhaiQPEzSFdw</code> to seed tick, liquidity, and sqrt price.
Replay event history (v2)	<code>download_uniswap_v2_events.py</code>	Allium Explorer [3]	Downloads replayable v2 event histories via query <code>4MykukH1JAzASTWB4Bdz</code> .
Replay event history (v3)	<code>download_uniswap_v3_events.py</code>	Allium Explorer [3]	Downloads replayable v3 event histories via query <code>uQXoH1F8G2TfJRT2Byk6</code> .
Tick initialization	<code>process_uniswap_v3_ticks.py</code>	Allium Explorer [3]	Pulls historical mint/burn ranges via query <code>Gpm54k46AGjtegifodfr</code> and reconstructs tick-liquidity net maps.

Component	Scripts	Verified source	Technical role
Fallback v3 state queries	<code>process_uniswap_v3_liquidity.py</code>	Ankr RPC endpoints	Uses committed Ankr endpoints for Arbitrum, Optimism, and Base to query <code>slot0()</code> and <code>liquidity()</code> when a pool receives a mint or burn before a swap has seeded in-memory state.
Single-chain detection logs	<code>detection/*/*single_chain_arbitrage.py</code>	Chain RPC providers imported from <code>utils.settings</code>	Direct <code>eth_getLogs</code> style retrieval through Web3. The exact detector provider brands are <i>not</i> committed, so naming Alchemy or Infura would be speculative.
Persistence and replay store	<code>process_*, simulate_paths.py, results/*</code>	Local MongoDB on <code>localhost:27017</code>	Stores normalized <code>dex_updates</code> and exported profitable-path or detection collections in database <code>cross_chain_arbitrage</code> .

This source split is important for interpretation. The *simulation* pipeline is primarily driven by Allium-exported histories replayed through MongoDB, whereas the *detection* pipeline uses direct RPC log scans and transaction receipts to reconstruct realized single-chain arbitrage.

3 Single-Chain Detection Methodology

3.1 What the detector actually searches for

Each chain-specific detector scans swap logs for two event signatures:

- the Uniswap v2 `Swap` topic `0xd78a...d822`;
- the Uniswap v3 `Swap` topic `0xc420...ca67`.

Events are grouped by block number and then by transaction index. Within a transaction, swaps are ordered by log index and examined as a possible cyclic sequence. This is *not* a triangle-only detector. A valid detected sequence may have any observed length greater than one. In the committed exports, realized detected paths span:

- 2 to 6 swaps on Arbitrum;
- 2 to 7 swaps on Base;
- 2 to 5 swaps on Optimism.

The candidate sequence $S = (s_1, \dots, s_m)$ must satisfy the repository’s exact continuity checks:

$$\text{token}_k^{\text{out}} = \text{token}_{k+1}^{\text{in}}, \quad (1)$$

$$\text{amt}_k^{\text{out}} \geq \text{amt}_{k+1}^{\text{in}}, \quad (2)$$

$$\text{pool}_k \neq \text{pool}_{k+1}, \quad (3)$$

for each adjacent pair (s_k, s_{k+1}) . A cycle closes when the first input token equals the current output token, with ETH and WETH treated as equivalent wrappers. The code also requires the first input amount to be no larger than the closing output amount. Therefore the detector is best described as *transaction-level cyclic swap reconstruction under token continuity constraints*.

3.2 Realized gain, cost, and profit

For a detected sequence, the code aggregates token balances across all included swaps. For token a ,

$$B_a = \sum_{j=1}^m \text{out}_{j,a} - \sum_{j=1}^m \text{in}_{j,a}.$$

Using token decimals d_a and the Allium price lookup $p_a(t)$ at the block timestamp t , the detector computes

$$G(t) = \sum_{a: B_a > 0} \frac{B_a}{10^{d_a}} p_a(t), \quad C_{\text{token}}(t) = \sum_{a: B_a < 0} \frac{-B_a}{10^{d_a}} p_a(t).$$

The realized profit is then

$$\Pi(t) = G(t) - C_{\text{token}}(t) - C_{\text{gas}}(t).$$

The gas term is chain-specific in code:

- **Arbitrum:**

$$C_{\text{gas,ETH}}^{\text{arb}} = \text{gasUsed} \cdot \text{gasPrice},$$

converted from wei to ETH and then into USD using the WETH price at the block timestamp.

- **Base and Optimism:**

$$C_{\text{gas,ETH}}^{\text{op-stack}} = \text{gasUsed} \cdot \text{effectiveGasPrice} + C_{L1},$$

where

$$C_{L1} = \begin{cases} \text{llGasUsed} \cdot \text{llGasPrice} \cdot \text{llFeeScalar}, & \text{if those receipt fields exist,} \\ \text{llFee}, & \text{otherwise.} \end{cases}$$

This treatment is consistent with OP Stack fee decomposition in the official documentation [6, 4], although the repository’s detector does not add any post-Isthmus operator fee field.

The practical implication is that the `results/detection/` exports already reflect *gas-adjusted realized single-chain profit*. They are therefore much closer to executable economics than the replay-based cross-rollup simulation outputs.

4 Path Construction and Replay Simulation

4.1 Reserve-token path construction

The file `simulation/path_builder.py` constructs a graph over AMM pools and reserve-token endpoints. Paths are capped by

$$\text{MAX_PATH_LENGTH} = 5.$$

Single-chain paths are cycles that start and end in the same reserve token. Cross-rollup paths are *pairwise* concatenations of two chain-local reserve-token paths. If $M_{c_1 \rightarrow c_2}$ denotes the repository’s symbol-based address mapping from chain c_1 to chain c_2 , the builder looks for path pairs satisfying

$$M_{c_1 \rightarrow c_2}(\text{end}(\pi^{(1)})) = \text{start}(\pi^{(2)}), \quad M_{c_2 \rightarrow c_1}(\text{end}(\pi^{(2)})) = \text{start}(\pi^{(1)}),$$

with $|\pi^{(1)}| + |\pi^{(2)}| \leq 5$.

That design implies two important facts:

- the implemented cross-rollup strategy class is *spatial cross-rollup arbitrage via reserve-token mapping*, not a coded bridge-transfer protocol;
- triangle arbitrage is only a special case among many possible cycle lengths, not the dominant description of the repository.

4.2 State initialization and replay store

The replay engine `simulation/simulate_paths.py` begins by loading:

1. precomputed path files;
2. `initial_pool_states.json`;
3. reserve-token price files under `data/prices/`;
4. incremental pool updates from MongoDB collection `dex_updates`.

For Uniswap v2, the state variables are the reserves (R_0, R_1) . For Uniswap v3, the code requires current tick, current sqrt price, active liquidity, and a tick-liquidity net map. The v3 updater combines swap, mint, and burn events and, when needed, falls back to direct RPC calls to `slot0()` and `liquidity()` at the relevant block.

Reserve-token prices are aligned by nearest previous timestamp:

$$p_{c,a}(t) = p_{c,a}(s^*), \quad s^* = \max\{s \leq t : s \in \mathcal{T}_{c,a}\}.$$

This is a backward-fill rule, not interpolation.

4.3 Gain screen and liquidity ranking

Before exact replay, the engine applies a route-level log-gain filter. For a Uniswap v2 route, the fee-adjusted directional price is

$$P^{(v2)} = \begin{cases} \frac{R_{\text{out}}}{R_{\text{in}}} 10^{d_{\text{in}} - d_{\text{out}}} (1 - 0.003), & \text{if zero-for-one,} \\ \frac{R_{\text{in}}}{R_{\text{out}}} 10^{d_{\text{out}} - d_{\text{in}}} (1 - 0.003), & \text{otherwise.} \end{cases}$$

For a Uniswap v3 route, the code first reconstructs the implied tick from the stored Q64.96 square-root price:

$$\tau = \log_{1.0001} \left(\frac{\text{sqrt_price}^2}{2^{192}} \right).$$

It then applies the directional price

$$P^{(v3)} = \begin{cases} 1.0001^\tau 10^{d_{\text{in}} - d_{\text{out}}} (1 - f), & \text{if zero-for-one,} \\ \left(1.0001^\tau 10^{d_{\text{out}} - d_{\text{in}}} \right)^{-1} (1 - f), & \text{otherwise,} \end{cases}$$

where $f = \text{fee_tier}/10^6$.

Each route contributes

$$g_k = -\log_2 P_k, \quad G(\pi) = \sum_k g_k,$$

and the path is retained only if

$$-1 < G(\pi) < 0.$$

The engine also computes a liquidity key per route. For v2 it uses $\sqrt{R_{\text{in}} R_{\text{out}}}$; for v3 it uses active liquidity. The implementation then rescales each route by a decimal-based factor and retains only the minimum such value as the path's liquidity bucket. Because the code stores decimal pairs asymmetrically by direction, this bucket should be interpreted as a coarse prioritization heuristic rather than a dimensionally exact liquidity measure.

4.4 Exact replay and slippage model

If every route in a path is Uniswap v2 and the total path length is 2, 3, 4, or 5, the engine uses closed-form optimization routines `fast_path_two_arb` through `fast_path_five_arb`. The underlying v2 swap function is exactly

$$\Delta y = R_{\text{out}} - \frac{R_{\text{in}} R_{\text{out}}}{R_{\text{in}} + 0.997 \Delta x} = \frac{0.997 \Delta x R_{\text{out}}}{R_{\text{in}} + 0.997 \Delta x}.$$

Otherwise, the engine uses `ternary_search(path)` on input size. The search starts at 0.1 units of the input token, expands the right boundary by powers of ten while the optimum remains near the boundary, and then performs a ternary search until the interval width is below 10% of the current

search range.

For Uniswap v3 swaps, the code steps across ticks. Let P denote the normalized square-root price and L the active liquidity. Then the directional token deltas used by the code are

$$\Delta x = L \left(\frac{1}{P_b} - \frac{1}{P_a} \right), \quad \Delta y = L(P_b - P_a),$$

with the sign determined by swap direction. Within a tick range, the next square-root price is updated by

$$P' = \begin{cases} \frac{LP}{L + \Delta x P}, & \text{zero-for-one,} \\ P + \frac{\Delta x}{L}, & \text{one-for-zero.} \end{cases}$$

The input amount is fee-adjusted first by $1 - f$, where f is the v3 fee tier. This is the repository’s exact slippage model for replay.

4.5 Conflict elimination and synchronization semantics

After replay, profitable paths are sorted by profit and greedily deconflicted: if a lower-ranked path shares any pool address with a higher-ranked one, it is discarded. The exported `total_profit_usd` for a window is therefore a sum of *non-conflicting* path profits, not the sum over every individually profitable path.

The repository supports two execution modes:

- **Single-chain mode:** iterates second by second, groups updates by block number within each second, and replays per updated block.
- **Cross-chain mode:** iterates over a user-specified `TIME_WINDOW` and replays all paths touched by updates whose timestamps fall in $(t_{i-1}, t_i]$.

The committed cross-chain exports all satisfy

$$\text{timestamp_range_stop} - \text{timestamp_range_start} = 2 \text{ seconds.}$$

Hence the stored cross-rollup study is a *2-second wall-clock synchronization experiment*. It is not a bridge-aware latency model.

One subtle implementation detail should be reported explicitly. In cross-chain mode, pool updates are deduplicated by pool address without an explicit sort on timestamp before the first seen record is retained. The code therefore approximates a representative per-pool state within each 2-second window, but it does not prove that the retained state is always the latest update in that window.

5 Empirical Results from the Committed Exports

All committed result files in `results/` correspond to **November 1, 2024 (UTC)**. The earliest stored simulation timestamp is 1730419200, which is 2024-11-01 00:00:00 UTC, and the detection

exports lie on the same date. No newer committed result artifacts are present in the repository.

5.1 Aggregate exported metrics

Table 2 reports the aggregate row-level outputs over the committed JSON exports.

Table 2: Aggregate metrics over committed November 1, 2024 exports. Detection *PnL rows* count only rows with numeric `total_profit_usd`.

Dataset	Files	Rows	PnL rows	Positive	Zero	Negative	Sum PnL (USD)	Mean PnL (USD)
Cross-chain simulation	5	43,199	43,199	34,886	8,313	0	1,764,433.18	40.8443
Single-chain simulation	13	122,401	122,401	13,716	108,685	0	890.98	0.007279
Detection: Arbitrum	1	5,409	5,235	4,176	0	1,059	1,297.31	0.2478
Detection: Base	3	22,860	21,905	17,090	0	4,815	27,610.34	1.2605
Detection: Optimism	1	7,890	7,846	7,741	0	105	113.08	0.0144

Three conclusions follow immediately.

1. **Cross-chain replay dominates single-chain replay in the stored simulation outputs.** The gap between \$1.764M and \$890.98 is too large to interpret naively as evidence of deployable superiority; it is more plausibly a consequence of the repository’s permissive 2-second synchronization rule and the absence of bridge settlement frictions.
2. **Realized single-chain detection is much smaller in scale but methodologically stronger.** Detection rows include transaction cost and can therefore be positive or negative.
3. **Base detection is extremely heavy-tailed.** Its mean net profit is \$1.2605, but that average is dominated by a handful of very large outliers.

5.2 What is actually profitable in the simulation exports

The profitable simulation exports are even more specific than Table 2 suggests.

- **All profitable simulation paths are Uniswap v3-only.** Although the engine supports v2 and v3 mechanics, every profitable path stored in both simulation export directories uses only v3 routes.
- **All profitable cross-rollup paths are pairwise, not three-rollup.** The profitable cross-chain opportunities split into 40,139 Arbitrum–Base paths, 9,382 Base–Optimism paths, and 8,175 Arbitrum–Optimism paths.
- **Path lengths are variable.** Profitable cross-rollup path lengths are 2: 10,819, 3: 25,963, 4: 12,560, and 5: 8,354. Profitable single-chain path lengths are 2: 204, 3: 6,536, 4: 3,726, and 5: 4,491. This directly contradicts any claim that the repository is only studying triangles.

At the opportunity level, the committed exports contain:

- 57,696 profitable non-conflicting cross-rollup opportunities, with mean profit \$30.58, median profit \$0.5207, mean input notional \$49,984.15, and median input notional \$1,536.20;
- 14,957 profitable non-conflicting single-chain opportunities, with mean profit \$0.05957, median profit \$0.01492, mean input notional \$163.82, and median input notional \$75.40.

The cross-rollup distributions are therefore highly skewed. For example, Arbitrum–Base opportunities sum to \$878,535.32, Base–Optimism opportunities sum to \$849,791.36, and Arbitrum–Optimism opportunities sum to only \$36,106.49. Mean profit per profitable opportunity is highest on Base–Optimism (\$90.58), followed by Arbitrum–Base (\$21.89), with Arbitrum–Optimism much smaller (\$4.42).

The replay engine is also selective. Averaged across stored windows:

- cross-chain replay touches 11,443.08 updated paths per window, keeps 354.86 positive-gain candidates, and simulates only 29.73 paths exactly;
- single-chain replay touches 5,597.22 updated paths per window, keeps 115.10 positive-gain candidates, and simulates only 2.38 paths exactly.

This large reduction shows that most computational work is in state propagation and coarse filtering, not in exact replay.

5.3 Detection economics and drawdown

Table 3 reports realized detection economics using the exported `total_gain_usd`, `total_cost_usd`, and `total_profit_usd` fields.

Table 3: Detected single-chain economics from committed exports. Max drawdown is computed on cumulative net profit after sorting by block timestamp.

Chain	Mean gain	Mean cost	Mean net	Median net	Success rate	Max DD
Arbitrum	\$0.3831	\$0.1318	\$0.2478	\$0.00646	79.77%	\$38.34
Base	\$1.6867	\$0.4109	\$1.2605	\$0.00466	78.02%	\$7.02
Optimism	\$0.04006	\$0.02561	\$0.01441	\$0.00130	98.66%	\$0.16

This table sharpens the interpretation considerably.

- **Base has the largest realized outliers, not the largest typical trade.** Its mean net profit is highest, but the median detected net profit is only \$0.00466. The gap between mean and median shows that the Base series is dominated by a small number of extreme observations, including a maximum detected trade of \$9,106.75.
- **Arbitrum is economically meaningful but less extreme.** Its average detected profit is \$0.2478, with a much larger cumulative drawdown (\$38.34) than Base or Optimism once trades are ordered through time.

- **Optimism is the smallest in absolute dollar terms.** It has the highest success rate, but its mean and median profits are both small.

The protocol mix in detection is also broader than in simulation. The most common realized patterns are $V3 \rightarrow V3$ and $V3 \rightarrow V3 \rightarrow V3$, but mixed $V2 \rightarrow V3$ and $V3 \rightarrow V2$ sequences are common on Arbitrum and Base. Therefore the repository’s single-chain detector is truly mixed-protocol, even though the profitable replay exports are v3-only.

5.4 Latency sensitivity: what the repository can and cannot support

The raw draft reserved space for a latency-sensitivity or window-sensitivity study. The code certainly exposes that capability: cross-chain replay is invoked as

```
python simulate_paths.py cross-chain START STOP TIME_WINDOW.
```

However, the committed result set does *not* contain a multi-window experiment. Every stored cross-chain export uses the same 2-second window. The academically correct statement is therefore:

The repository currently supports latency-sensitivity analysis in principle, but the committed outputs do not yet contain a multi-window sweep. Any reported latency-sensitivity curve would require new runs, not reinterpretation of the existing exports.

One related point is nonetheless measurable from the stored data. Because the simulation exporter stores only non-negative row-level profits (\$0 or greater), cumulative row-level drawdown for the committed simulation series is zero by construction. That is a property of the export format and replay logic; it should *not* be interpreted as evidence of real execution robustness.

6 Limitations and Reproducibility Notes

The repository already contains a serious research pipeline, but several limitations should be stated plainly in a submission-ready report.

6.1 No bridge-aware execution layer

Cross-rollup paths are created by reserve-token address mapping, not by an explicit bridge object. There is no coded representation of:

- canonical bridge delay;
- fast-bridge fees;
- bridge failure probability;
- inventory pre-positioning across rollups.

Consequently, the cross-chain simulation outputs should be interpreted as *state-consistent upper bounds under a 2-second synchronization rule*, not as deployable cross-rollup P&L.

6.2 Cross-rollup synchronization is coarse

Single-chain replay respects block-local update order because the updater groups records by block number before replay. Cross-chain replay is coarser: it uses wall-clock windows and a first-seen per-pool update within each window. This is a reasonable research approximation, but it is not equivalent to executable cross-domain settlement.

6.3 Provider configuration is only partially reproducible from the repository

The detection scripts import their provider constants from `utils.settings`, but that file is not committed. Therefore the report can verify that the detectors use chain RPC endpoints, but it cannot truthfully claim that those endpoints are Alchemy, Infura, or any other brand. The only directly verifiable RPC brand in the committed repository is Ankr, used in the Uniswap v3 liquidity fallback updater.

6.4 Date parameters are not fully normalized across scripts

Most of the replay pipeline is parameterized around November 1, 2024, but the committed download scripts do not all share the same hard-coded date constants. In particular, the v2 and v3 event download scripts contain later default dates than the committed result exports. Reproducing the exact stored experiments therefore requires manually aligning those date constants before rerunning the pipeline.

6.5 “Trust Capital” is not implemented

The raw draft discussed a reliability score under the label “Trust Capital.” No such score is computed anywhere in the committed code. It is acceptable to describe this as future work, but not as implemented functionality.

7 Conclusion

After grounding the draft in the repository, the most defensible characterization of the project is this: it is a robust Python research framework for *single-chain realized arbitrage detection* and *pairwise cross-rollup opportunity replay*, built around Uniswap v2/v3 mechanics, Dune/Allium data ingestion, and MongoDB-backed incremental state updates.

The core theoretical correction is straightforward. The repository is not a narrow triangle-arbitrage prototype. It detects variable-length single-chain cyclic swap sequences from real transactions and simulates reserve-token cycles of length up to five, including pairwise spatial cycles across Arbitrum, Base, and Optimism. The strongest empirical result is also straightforward: the committed 2-second cross-rollup replay produces many large theoretical opportunities, but those opportunities remain materially overstated until bridge timing, inventory, and competition are modeled explicitly. By contrast, the detection exports are economically smaller yet much more credible because they already include transaction-cost accounting.

The next research step is therefore not to inflate the current cross-rollup profit totals. It is to tighten the execution model: add a bridge layer, run a genuine multi-window sensitivity study, normalize the provider configuration for reproducibility, and keep proposed reliability metrics clearly separated from implemented results.

References

- [1] Hayden Adams, Noah Zinsmeister, and Dan Robinson. Uniswap v2 core, 2020. <https://docs.uniswap.org/whitepaper.pdf>. Accessed 2026-03-27.
- [2] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v3 core, 2021. <https://uniswap.org/whitepaper-v3.pdf>. Accessed 2026-03-27.
- [3] Allium. Allium explorer, 2026. <https://www.allium.so/product/allium-explorer>. Accessed 2026-03-27.
- [4] Base. Network fees, 2026. <https://docs.base.org/base-chain/network-information/network-fees>. Accessed 2026-03-27.
- [5] Dune. Get latest query result, 2026. <https://docs.dune.com/api-reference/executions/endpoint/get-query-result>. Accessed 2026-03-27.
- [6] Optimism. Transaction fees on op mainnet, 2026. <https://docs.optimism.io/op-stack/transactions/fees>. Accessed 2026-03-27.